

Software Testing Report

SE3110: Quality Management in Software Engineering, Year 3 Semester 1, 2026 Software Testing and Quality Evaluation of the SE3090 Integrated System: AgriConnect

System under test	AgriConnect: ASP.NET Core 10 API, PostgreSQL, React web app, Flutter mobile app, Python Agentic AI service
Test period	3 to 8 October 2026 (final full run on 7 October 2026)
Group members	Adithya M A D K (IT24102690), Marasinghe M A Y D (IT24102714), Wijekoon W H M P V P (IT24103113), Subasingha S A P R (IT24103048)
Companion documents	Test Plan, Test Case Document, Defect Report, Executed Test Results (all in documentation/testing/)
Evidence	testing-evidence/run-2026-10-07/ and testing-evidence/web/ in the repository

1. Summary

All five automated suites pass on the final run: **523 tests, 0 failures** (xUnit 378, pytest 89, Vitest 24, Playwright 12, flutter_test 20). Four non-functional tools were run on the real system: k6 (performance), OWASP ZAP and scripted checks (security), and Lighthouse (accessibility). One complete business workflow was driven through every layer: React UI, API, PostgreSQL and the Agentic AI service.

Testing found real defects, and the important ones were fixed and retested:

Finding	Result
No limit on failed logins (brute force possible)	Fixed: lockout after 5 failures; 13 of 13 security checks pass
API answered HTTP 500 when the database was saturated	Fixed: 503 with Retry-After ; 702 of 702 requests were 500 before, 750 of 750 are 503 after
X-Content-Type-Options header missing (ZAP)	Fixed: ZAP warnings 4 to 2
Dashboard filters had no accessible name (Lighthouse 89)	Fixed: score 93
API unusably slow against the cloud database (15-connection limit)	Open: needs a connection-string change; the same code answers in 36 ms (p95) on a local database

Of the 105 planned test cases, 77 were executed and passed and 28 were not run (mobile device tests, Firefox/WebKit/Edge, direct database-constraint tests and some performance shapes). They are listed with reasons in section 9; none is hidden in the pass rate.

2. Scope and objectives

Objectives: show that the features and workflows of the four components work as the SRS specifies; that stock reservation never oversells under concurrency (FR9); that no AI proposal is committed without a human decision (FR19, NFR Safety); that the API stays responsive and secure under load and attack; and that defects are found, fixed and retested.

In scope: backend API and business logic, database behaviour, React web app, Flutter mobile app, cross-component workflows, performance, security, accessibility, compatibility and the Agentic AI service. Out of

scope: real payments (not part of the product), penetration testing beyond ZAP's automated scan, and load tests of third-party services (Maps and LLM providers were stubbed or run in mock mode).

3. Test strategy

The strategy follows the testing pyramid, with the riskiest behaviour tested at the cheapest level that can show it:

Risk	Where it is tested	Tool
Overselling stock under concurrency (FR9)	Service tests with parallel reservations on real PostgreSQL; then 60 simultaneous HTTP orders	xUnit, k6
Wrong or unsafe order transitions, role leaks, IDOR	Service and API tests through the real HTTP pipeline; scripted attacks	xUnit <code>WebApplicationFactory</code> , security script
AI output trusted blindly, prompt injection, auto-commit	Deterministic agent tests with fake models; workflow test shows slot stays Proposed until an officer confirms	pytest, Playwright
UI forms, route protection, error and empty states	Component tests; browser tests	Vitest, Playwright, <code>flutter_test</code>
Components drifting apart after independent development	One full workflow across UI, API, database and AI	Playwright
Slow or failing API under load; vulnerabilities; accessibility	Load, stress and race scenarios; authenticated scan; page audits	k6, ZAP, Lighthouse

Performance and security are the required non-functional types. Accessibility was added because farmers and buyers vary in device quality and ability. Compatibility was planned but only Chromium could be run.

4. Test environment

Item	Value
Machine	Windows 11 laptop, Docker Desktop for k6 and ZAP images
API	.NET 10, Release build, <code>localhost:5000</code> , Development environment (seeded)
Database	PostgreSQL on localhost, dedicated database <code>agriconnect_perf</code> (migrated from scratch and seeded). The earlier backend run and k6 run A used the configured cloud database
Web	React dev server <code>:3000</code> ; Chromium 1228 (Playwright 1.63)
AI	agentic-ai FastAPI on <code>:8000</code> , <code>LLM_PROVIDER=mock</code> (no external LLM)
Tools	xUnit 2.9, pytest 9, Vitest 4, Playwright 1.63, <code>flutter_test</code> (Flutter 3.47), k6 (Docker <code>grafana/k6</code>), OWASP ZAP 2.17 (Docker), Lighthouse

5. Execution summary

Suite	Tool	Executed	Passed	Failed
Backend unit, service, API integration, concurrency	xUnit	378	378	0
Agentic AI (including 6 safety cases)	pytest	89	89	0

Suite	Tool	Executed	Passed	Failed
Web components	Vitest + React Testing Library	24	24	0
Web end-to-end and integrated workflow	Playwright (Chromium)	12	12	0
Mobile widgets and API client	flutter_test	20	20	0
Total		523	523	0

Coverage where measured: Agentic AI 67% of lines; Vitest 60% of the files it targets; Flutter lcov report saved. Backend coverage was not measured.

Planned test cases (Test Case Document, Part E): 105 planned, 77 executed and passed, 0 failed, 28 not run.

First runs were not clean, and the failures were recorded as defects instead of being hidden: 4 of 83 pytest tests, 9 of 15 Vitest tests, 8 of 10 Playwright tests, and one suite did not load. All were test-infrastructure problems (a developer's `.env` leaking into tests, undeclared test dependencies, tests older than the UI), not product bugs. The product defects came from the non-functional tools.

6. Integrated workflow testing

`web/tests/e2e/workflow.spec.ts` (XP-W-01, XP-W-02, XP-W-05b) runs one real business flow with two browser sessions:

1. A buyer signs in, opens a listing, and orders 5 kg in the browser; the UI confirms "reserved".
2. The API shows the order as Pending and the listing's available stock reduced by exactly 5 (UI and database agree).
3. An officer signs in, opens the order and approves it. The API calls the Agentic AI logistics agent (`POST /agents/logistics/schedule` returned 200 in the AI service log) and a pickup slot is **proposed**, not confirmed.
4. The officer reviews and confirms the slot; the order becomes Scheduled, then Completed.
5. The audit trail has all five events (`OrderCreated`, `OrderStatusChanged`, `SchedulePropose`, `ScheduleDecision`, `OrderStatusChanged`), and the buyer was notified.

A second test shows that a buyer who types `/orders/{id}` gets "Access denied". Evidence: `run-2026-10-07/xp-w-01-evidence.txt`. Result: **Passed**.

7. Non-functional testing

7.1 Performance (k6)

Script: `testing-evidence/performance/agriconnect-performance.js`. Scenarios: browse load (20 VUs, 30 s), browse stress (ramp to 200 VUs), login (10 VUs), and an order race (60 orders from 30 VUs against one listing whose stock, 370 kg, is smaller than the demand). Thresholds: p95 under 500 ms and under 1% errors at normal load, plus the stock arithmetic checked afterwards.

Run	Database	Browse p95	Login p95	Failed browse requests	Checks	Order race
A (3 min)	Cloud (15-client pooler)	4.84 s	15.04 s	28.2%	22.5% passed	15 accepted, 30 errors; stock arithmetic still correct
B	Local PostgreSQL	38.8 ms	166 ms	0%	100%	37 accepted, 23 refused

Run	Database	Browse p95	Login p95	Failed browse requests	Checks	Order race
C (after fixes)	Local PostgreSQL	36.0 ms	175 ms	0%	100% (74,789)	37 accepted, 23 refused

Interpretation:

- **FR9 holds under HTTP load.** With 370 kg available and 10 kg orders, exactly 37 orders (370 kg) were accepted and 23 refused; stock ended at 0 and never went negative. In run A, even while the database was failing, no oversell happened.
- The application is fast: about 420 requests per second at p95 36 ms, with no errors up to 200 virtual users.
- Run A exposed two defects: the 500 responses (DEF-P-01, fixed) and the connection-limit mismatch (DEF-P-02, open).
- Run A saturated the shared cloud database's pool for about three minutes. It was not repeated.

Retest of DEF-P-01 (`performance/pool-exhaustion.js`): with the database unreachable, 702 of 702 requests answered 500 before the fix; after it, 750 of 750 answered 503 with `Retry-After: 5`.

7.2 Security (OWASP ZAP and scripted checks)

- **ZAP API scan** from the OpenAPI spec (84 URLs, 18,422 requests, active scan, authenticated as an officer): **0 High, 0 Medium, 4 Low**. After fixes a passive re-scan shows 2 Low: missing `Cross-Origin-Resource-Policy` (accepted, images are cross-origin) and a "timestamp disclosure" false positive.
- **Scripted checks** (`testing-evidence/security/security_checks.py`, 13 checks): missing and tampered tokens (401), role misuse (403), IDOR on orders (404/403), SQL injection in search and sort, malformed input leaks no stack trace, CORS rejects an arbitrary origin, client-supplied price and status ignored on order creation, and brute-force throttling. First run: 12 passed, 1 failed (no throttling, DEF-S-01). After the fix: 13 of 13.
- ZAP's scan was limited to the officer role; Administrator-only endpoints were not scanned with admin rights.

7.3 Accessibility (Lighthouse)

Page	Before	After fixes
Login	100	100
Farmer dashboard	89	93
Buyer dashboard	89	93
Price trends (officer)	100	100
AI scheduling (officer)	100	100

The six unlabelled filter dropdowns were labelled (DEF-W-01). Remaining audits on the dashboards (contrast, heading order, notification button name) are recorded as DEF-W-02.

7.4 Compatibility and other types

Only Chromium could be run (all 12 browser tests pass). Firefox, WebKit and Edge were not run because browser downloads failed when the system drive filled up. Usability testing was not performed. Reliability was tested through the database-failure scenario (DEF-P-01).

8. Agentic AI testing and evaluation

89 pytest cases (67% line coverage) cover: the four required golden scheduling cases; structured-output validation (7 malformed model replies all rejected); tool behaviour and tool failures; matching business rules (closest centre with capacity, full centres skipped, no match when all are full); guardrails on the model's explanation text; internal API key enforcement; and failure recovery (model down gives a 502 for scheduling and a deterministic fallback for matching).

Six safety cases were added: an injected instruction in a data field is not obeyed; a model that does obey it is caught by validation; neither agent's output contract has any field that could commit a schedule or a match; a misleading narration is discarded; and the agent still answers safely when the model is down. At system level the integrated workflow shows the slot stays Proposed until an officer confirms it.

All tests use a mock or fake model, so they are deterministic. No real LLM quality evaluation (for example scoring free-text reasoning) was done.

9. Not run, and limitations

- Mobile `integration_test` and Appium flows (no emulator; there is no `integration_test` suite in `mobile/`).
- Firefox, WebKit and Edge runs.
- Direct database constraint tests (CHECK, NOT NULL, foreign key, cascade, forced rollback) were not written; behaviour is covered indirectly through services and APIs.
- Performance: no 1-VU baseline, spike or separate recovery phase; scale was lower than the plan (20 VUs normal, 200 stress) and used a local database.
- Farmer listing creation through the browser, registration validation, and dynamic XSS testing.
- Backend code coverage was not measured.
- DEF-D-02 and DEF-D-03 (Component D UI) were fixed, but the retest screenshots named in the manual log are not in the repository.

10. Defect summary

13 defects were logged against the system (plus 4 test-infrastructure defects). 9 fixed and retested, 1 open (configuration), 1 open (low), 2 accepted (low). Details, steps to reproduce, evidence and retest results are in the Defect Report.

Severity	Found	Fixed	Open / accepted
High	6	5	1
Medium	3	3	0
Low	4	1	3

11. Conclusion

Within what was run, the system is functionally sound and its core guarantee holds: stock is never oversold under concurrent HTTP load, AI output is only ever a proposal, role and ownership checks hold, and the API is fast on a properly provisioned database. The most valuable results came from the non-functional tools: a missing brute-force defence, the wrong HTTP status for database overload, and a deployment limit that makes the cloud configuration much slower than the code. The first two are fixed and retested.

The third needs a connection-pool setting before the application is used by more than a handful of people on the current cloud database. The unexecuted areas (mobile devices, other browsers, database constraints) are the main remaining risk.

12. How to rerun

```
# Backend (set the connection string to a throwaway database)
ConnectionStrings__Default="Host=localhost;Database=agricconnect_perf;Username=postgres;Password=postgres"
dotnet test backend.Tests
# Agentic AI
cd agentic-ai && uv sync --extra dev && uv run python -m pytest tests --cov=src
# Web
cd web && npm install && npx vitest run --coverage
CHROMIUM_PATH=<path to chrome.exe, optional> npx playwright test --project=chromium # needs API
:5000, web :3000, AI :8000
node tests/all/lyighthouse.mjs
# Mobile
cd mobile && flutter test --coverage
# Performance and security (API on :5000, seeded throwaway database)
docker run --rm -i --add-host=host.docker.internal:host-gateway -e
BASE_URL=http://host.docker.internal:5000 grafana/k6 run - < testing-evidence/performance/agricconnect-
performance.js
uv run --project agentic-ai python testing-evidence/security/security_checks.py
# Executed-results appendix
python testing-evidence/build_results_appendix.py
```

13. Use of AI

AI assistance (Claude Code) was used to help run the suites, diagnose failures, write test scripts (k6, security checks, the integrated Playwright workflow, the Agentic AI safety cases), implement the fixes for DEF-P-01, DEF-S-01, DEF-S-02 and DEF-W-01, and draft these documents. Every script and result was run against the real system and the numbers in this report come from the saved tool output. Each member must be able to explain and rerun the tests in their own area at the viva.

The module's CLEAR-framework declaration is to be completed by each member in the form the module requires.